

AI Practical 4

1.Using Branch & Bound (optimized)

```
package aii

import java.util.Scanner;

public class n_queen {

static int N;

static int board[][];

static boolean col[];

static boolean diag1[];

static boolean diag2[];

static boolean solve(int row) {

if (row == N)

return true;

for (int c = 0; c < N; c++) {

if (!col[c] && !diag1[row + c] && !diag2[row - c + N -

1]) {

board[row][c] = 1;

col[c] = true;

diag1[row + c] = true;

diag2[row - c + N - 1] = true;

if (solve(row + 1))

return true;

board[row][c] = 0;

col[c] = false;

diag1[row + c] = false;

diag2[row - c + N - 1] = false;

}

}

return false;

}
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter value of N: ");
    N = sc.nextInt();
    board = new int[N][N];
    col = new boolean[N];
    diag1 = new boolean[2 * N];
    diag2 = new boolean[2 * N];
    if (solve(0)) {
        System.out.println("Solution:");
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                System.out.print(board[i][j] + " ");
            }
            System.out.println();
        }
    } else {
        System.out.println("No Solution exists");
    }
}

```

```

<terminated> n_queen [Java Application] C:\Users\HEMANT\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20;
Enter value of N: 8
Solution:
1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0

```

2. N-Queens using Backtracking

```
package aii;

import java.util.Scanner;

public class Backtracking {

    static int N;

    static boolean isSafe(int board[][], int row, int col) {

        for (int i = 0; i < row; i++)
            if (board[i][col] == 1)
                return false;

        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
            if (board[i][j] == 1)
                return false;

        for (int i = row, j = col; i >= 0 && j < N; i--, j++)
            if (board[i][j] == 1)
                return false;
        return true;
    }

    static boolean solve(int board[][], int row) {
        if (row == N)
            return true;

        for (int col = 0; col < N; col++) {
            if (isSafe(board, row, col)) {
                board[row][col] = 1;
                if (solve(board, row + 1))
                    return true;
                board[row][col] = 0; // Backtrack
            }
        }
    }
}
```

```

    return false;
}

static void printBoard(int board[][]) {
    System.out.println("Solution:");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            System.out.print(board[i][j] + " ");
        System.out.println();
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter value of N: ");
    N = sc.nextInt();
    int board[][] = new int[N][N];
    if (solve(board, 0))
        printBoard(board);
    else
        System.out.println("No Solution");}
}

```

The screenshot shows the Eclipse IDE's console window. The title bar indicates the application is a Java Application. The console output shows the user entering '4' for N, followed by the program's output of a 4x4 board solution. The board is represented as a grid of 0s and 1s, where 1 indicates a queen is placed in that row and column.

```

<terminated> Backtracking (1) [Java Application] C:\Users\HEMANT\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21
Enter value of N: 4
Solution:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

```